CMPS453 Cloud Computing System Models & CAP Theory

Ayman AboElHassan, PhD
Assistant Professor
ayman.abo.elmaaty@eng.cu.edu.eg

Today

2 Thought experiments

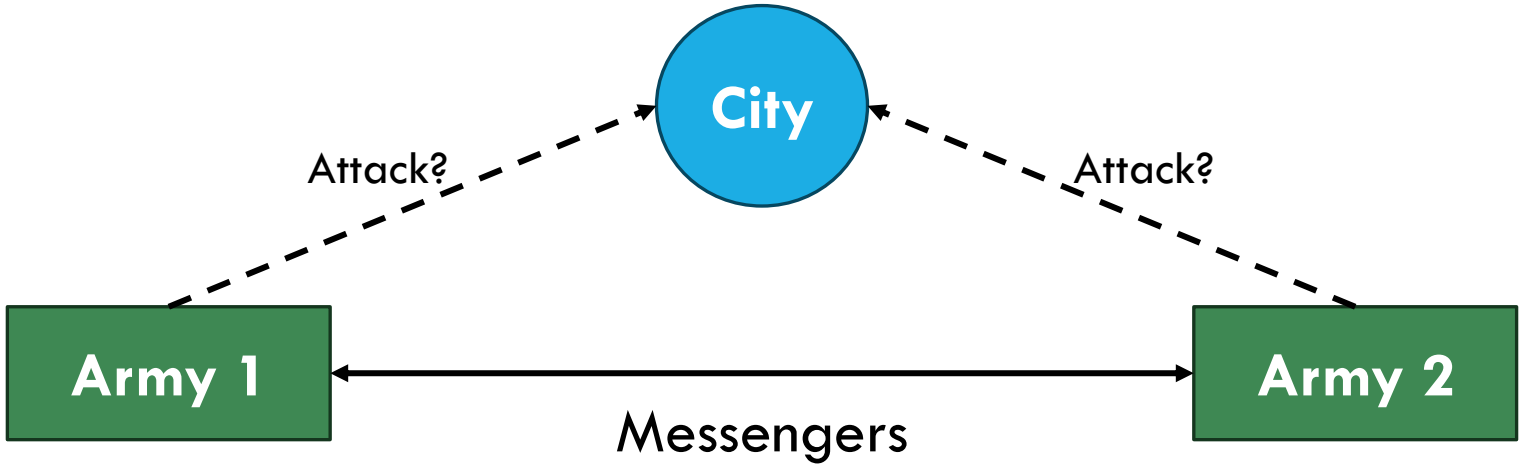
- 2 Generals Problem
- Byzantine Generals Problem

Breakdown the System

- System Model
- System Failure
- Replications
- CAP Theory

Two Generals Problem

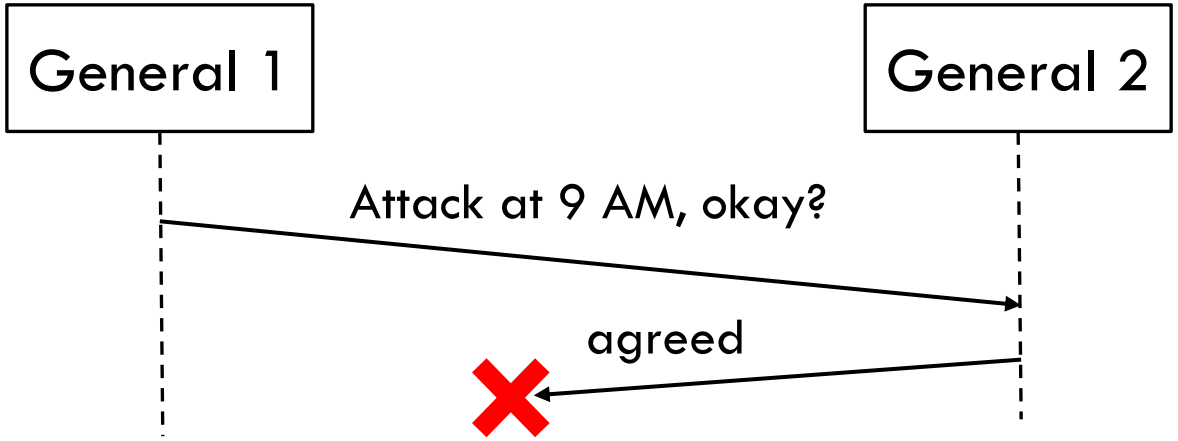
The Two Generals Problem



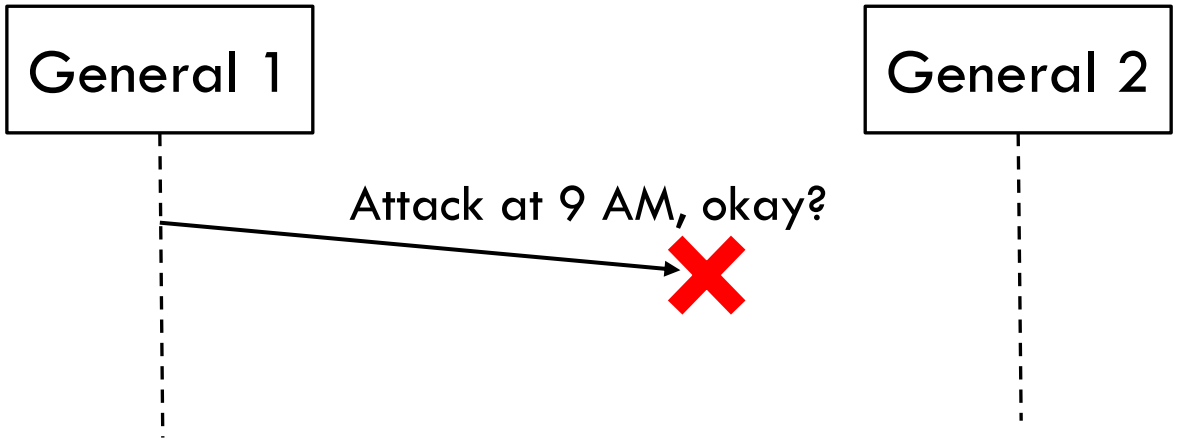
Army 1	Army 2	Outcome
Not Attack	Not Attack	Nothing Happens
Attack	Not Attack	Army 1 Defeated
Not Attack	Attack	Army 2 Defeated
Attack	Attack	City Captured

The Two Generals Problem

Case 1



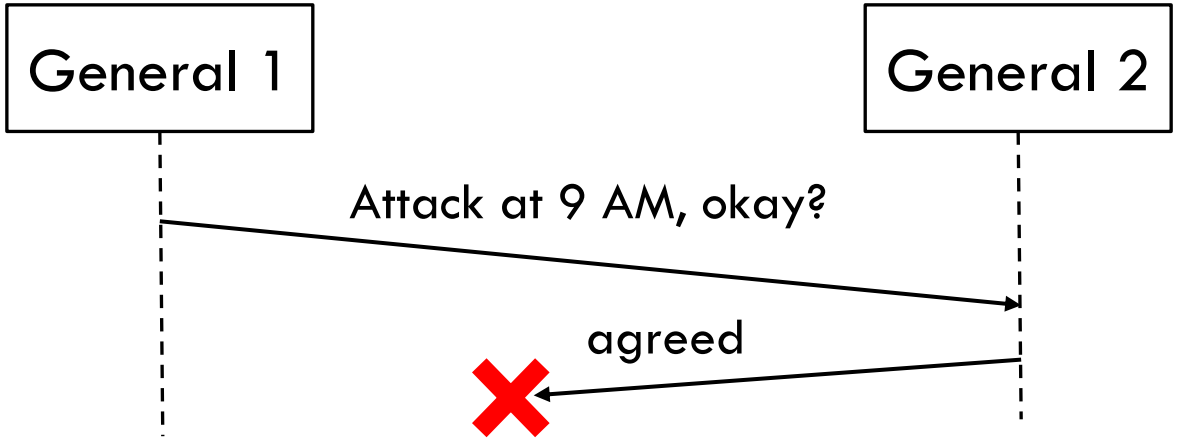
Case 2



The Two Generals Problem

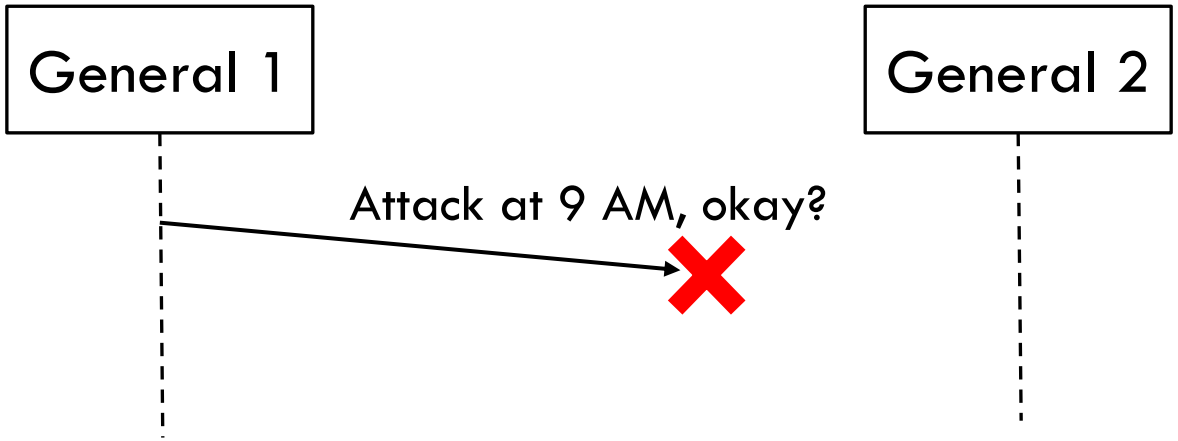
Case 1

General 1
- No response
General 2
- Agrees to attack



Case 2

General 1
- No response
General 2
- Doesn't know about the attack



The Two Generals Problem

Finding a solution:

1. General 1 will always attack
   - General 1 sends lots of messengers
     - If one gets through ➔ both generals attack ➔ Win
     - If all are captured ➔ General 2 doesn't know ➔ General 1 loses

The Two Generals Problem

Finding a solution:

1. General 1 will always attack
   - General 1 sends lots of messengers
     - If one gets through ➔ both generals attack ➔ Win
     - If all are captured ➔ General 2 doesn't know ➔ General 1 loses
2. General 1 only attacks if positive response received
   - General 1 is safe if no msg reaches General 2
   - If General 2 knows ➔ Must send response ➔ Situation1 but General 2

The Two Generals Problem

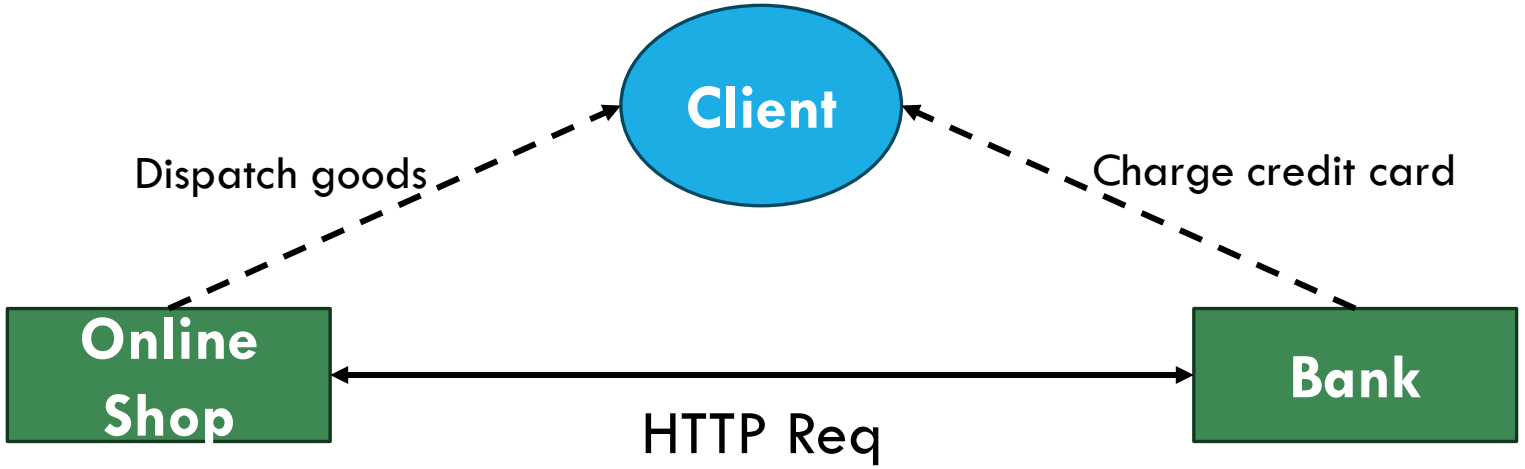
No common knowledge

The only way to know something is to communicate it

Finding a solution:

1. General 1 will always attack
   - General 1 sends lots of messengers
     - If one gets through ➔ both generals attack ➔ Win
     - If all are captured ➔ General 2 doesn't know ➔ General 1 loses
2. General 1 only attacks if positive response received
   - General 1 is safe if no msg reaches General 2
   - If General 2 knows ➔ Must send response ➔ Situation1 but General 2

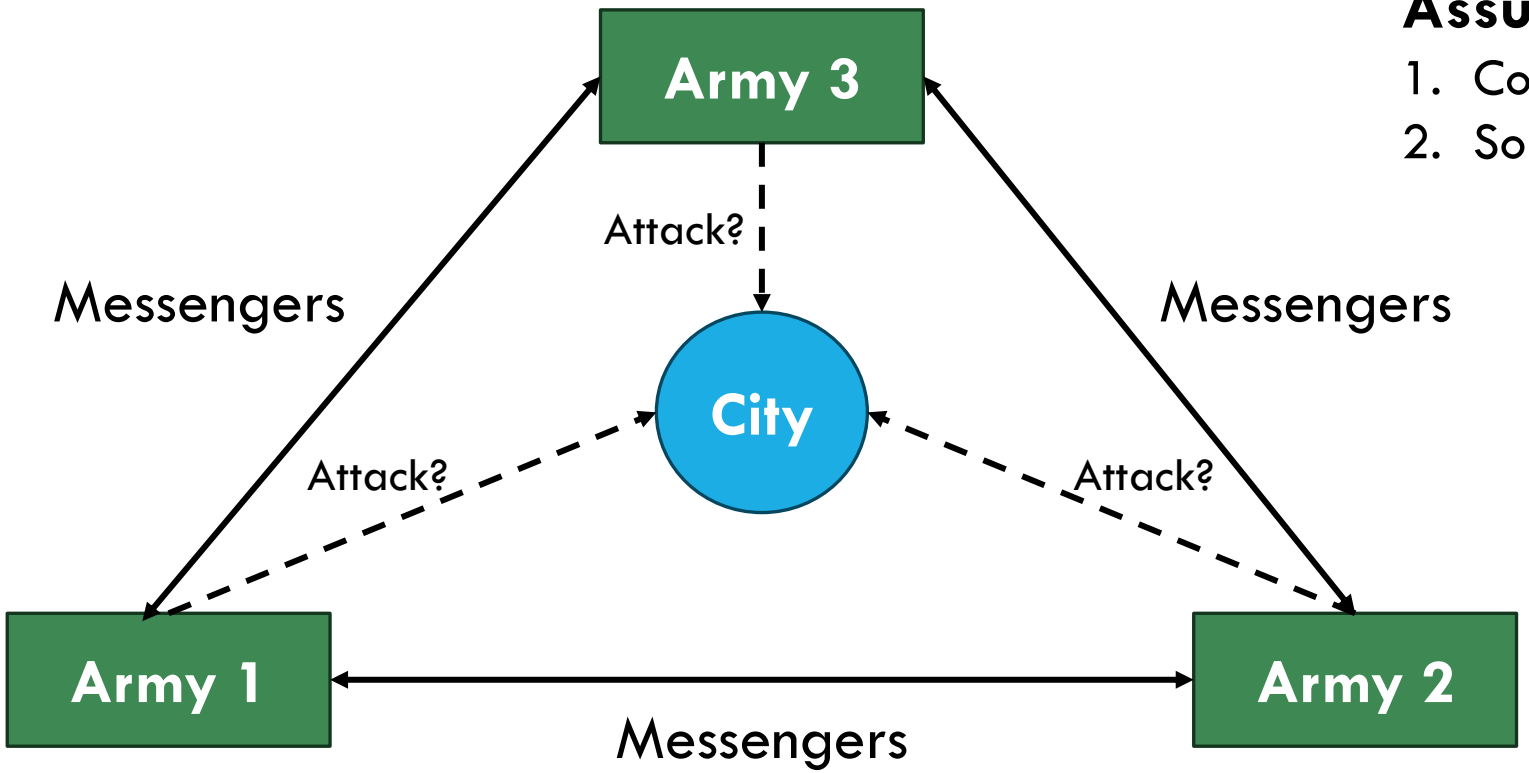
The Two Generals Problem applied



Online Shop	Bank	Outcome
Does not dispatch	Does not charge	Nothing Happens
Dispatches	Does not charge	Shop loses money
Does not dispatch	Charges	Customer complaint
Dispatches	Charges	Everyone happy

Byzantine Generals Problem

The Byzantine Generals Problem

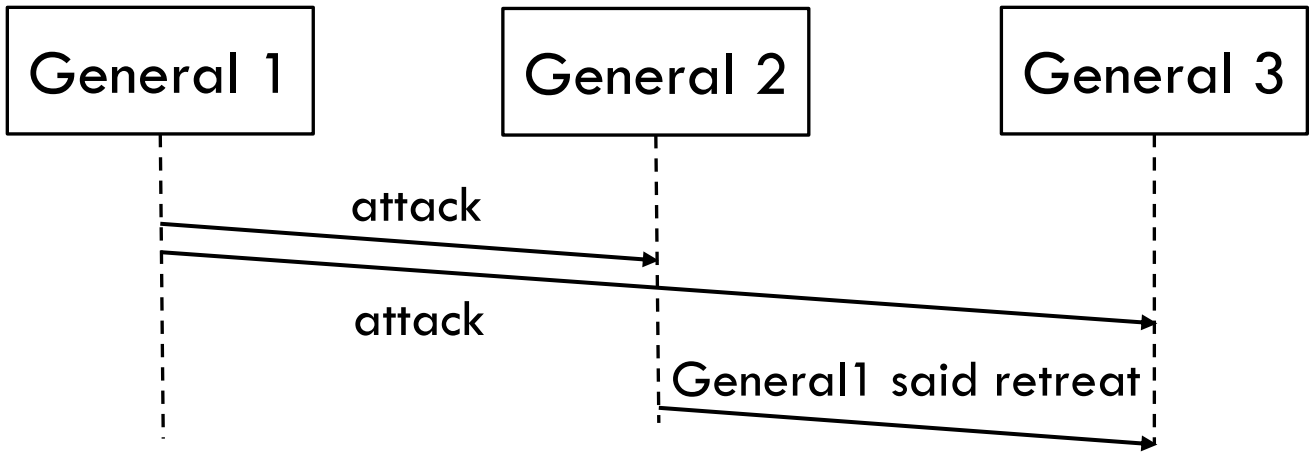


Assumptions:

1. Communication is reliable
2. Some generals might be traitors

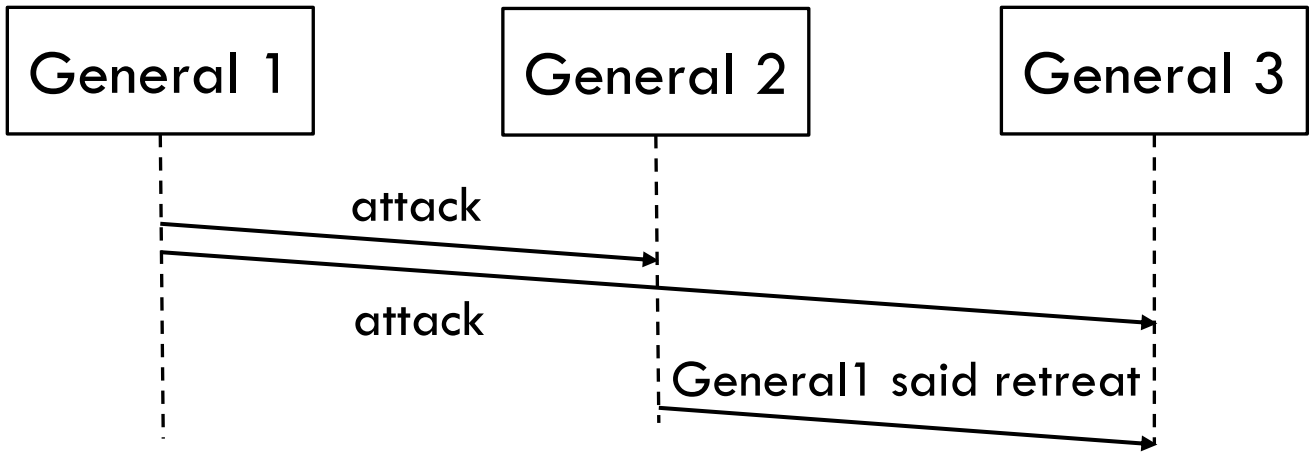
The Byzantine Generals Problem

Generals might lie

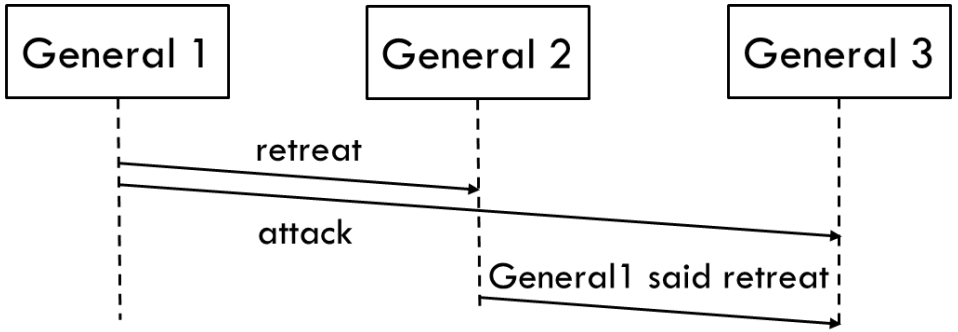


The Byzantine Generals Problem

Generals might lie



- From General 3 point of view
  - General 2 might be lying
  - Or general 1 might be lying ➔
    - General 2 is repeating honestly



The Byzantine Generals Problem

Assume

- Up to f generals are malicious
- Honest generals do not know who the malicious ones are
- The malicious generals may collude

Goal ➔ Honest generals must agree on a plan

The Byzantine Generals Problem

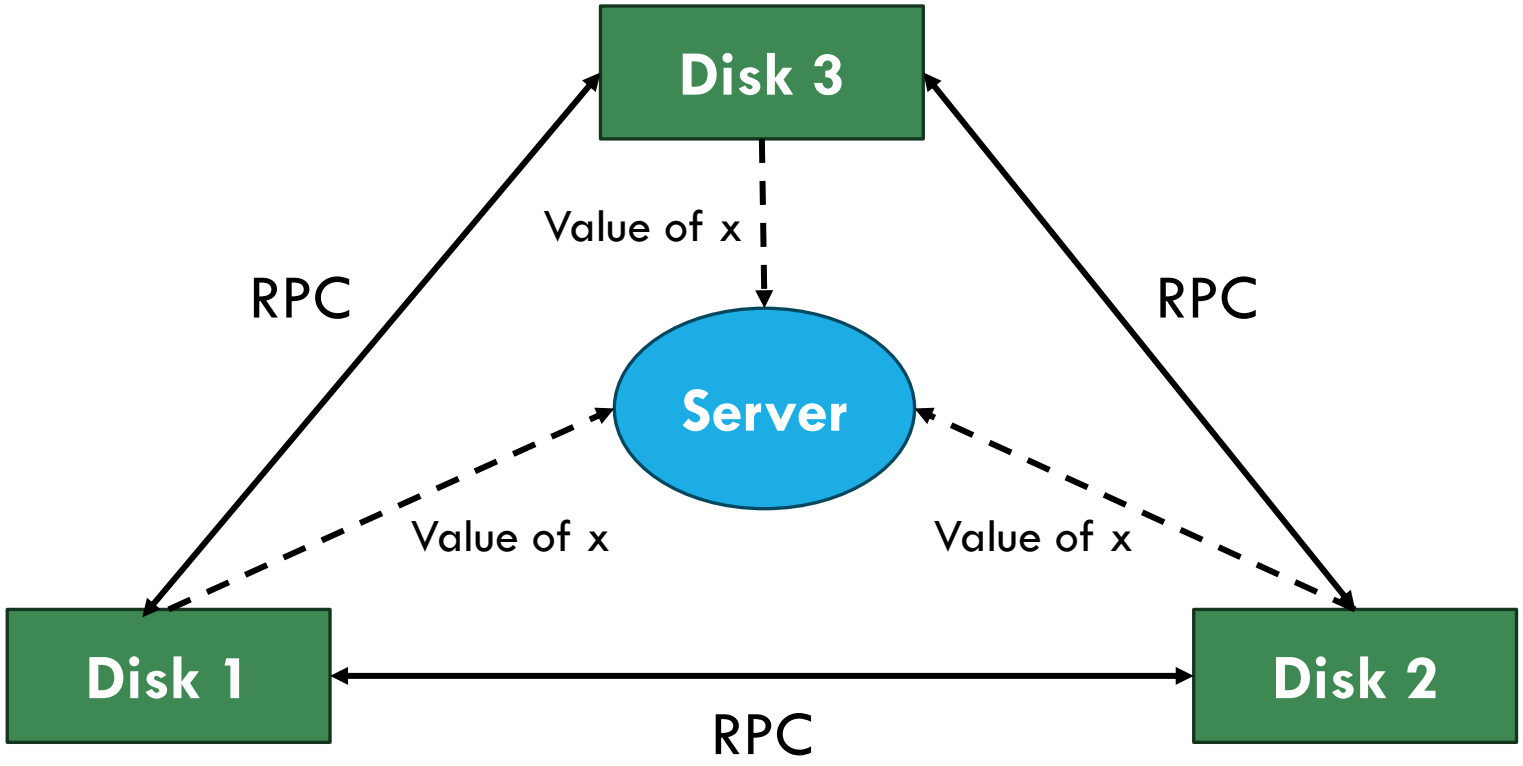
Assume

- Up to f generals are malicious
- Honest generals do not know who the malicious ones are
- The malicious generals may collude

Goal ➔ Honest generals must agree on a plan

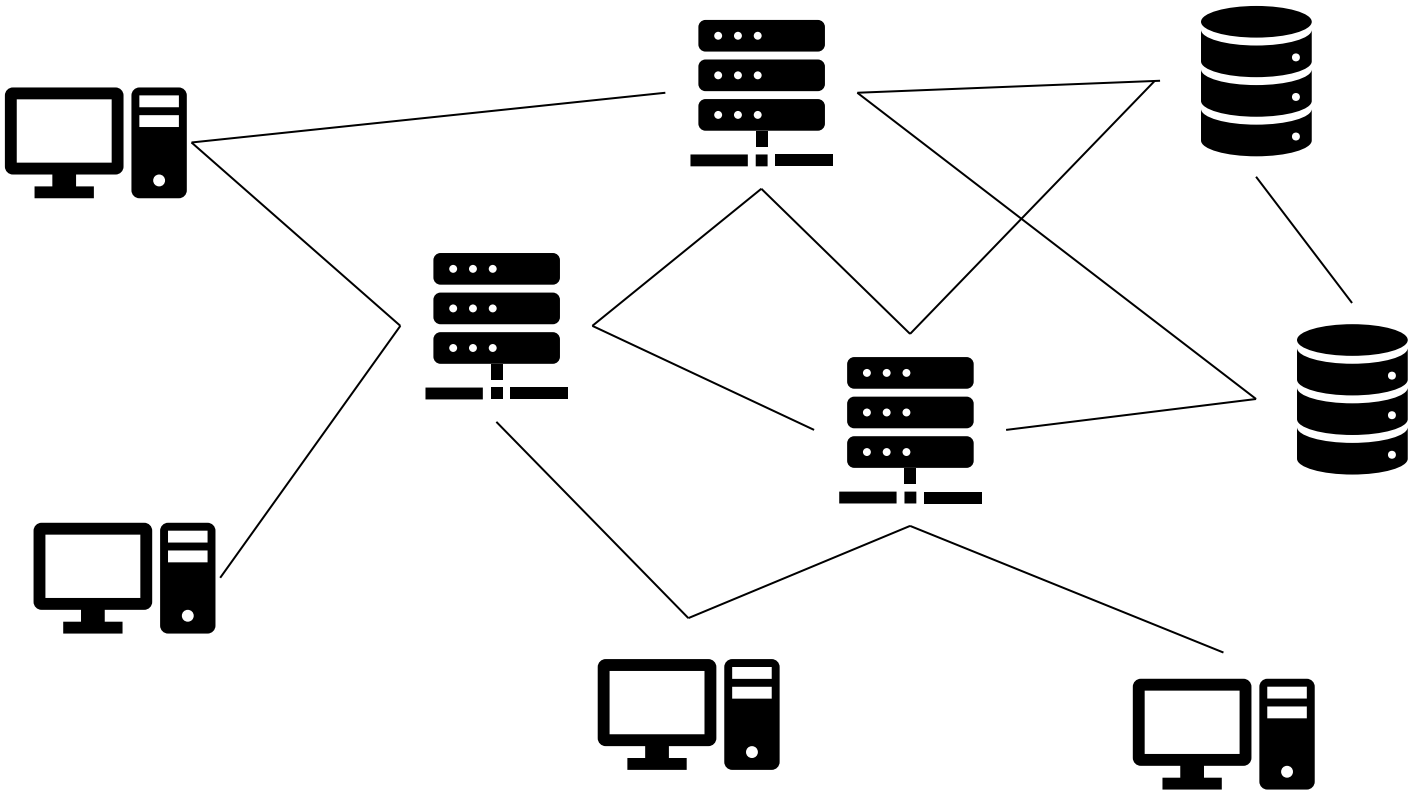
<u>Theorem:</u> We need at least $3f + 1$ generals in total to tolerate f malicious generals (i.e. $< \frac{1}{3}$ may be malicious)

The Byzantine Generals Problem Applied



System Model

Distributed System Components



Nodes

Data Links

Time

Distributed System Components

2 Generals problem

- Nodes are honest
- Messages can be lost (faulty network)

Byzantine generals problem

- Nodes are malicious (faulty node behavior)
- Messages are reliable

Real Systems

- Nodes & messages are faulty

Distributed System Components

We need to define our assumptions about the system

1. Network Behavior (ex: message loss)
2. Node Behavior (ex: crashes)
3. Timing Behavior (ex: sync/async)

1. Network Behavior

Assuming bidirectional **point-to-point** comm between 2 nodes

1. **Reliable** (perfect) links
   - Messages always goes through
   - No messages lost or fabricated, but it may be reordered
2. **Fair-loss** links
   - Messages have non-zero probability of going through
   - Messages may be lost, duplicated, or reordered
   - If we keep retrying, a message will eventually get through

1. Network Behavior

Assuming bidirectional **point-to-point** comm between 2 nodes

3. **Arbitrary** (adversary) links
   - A malicious active adversary may modify the link traffic
   - Interferes with messages (eavesdrop, modify, drop, spoof, replay)
   - Example: the real internet

Network partition

- Nodes are running fine
- Some links are interrupted for a while (Messages dropped/delayed)

1. Network Behavior

Assuming bidirectional **point-to-point** comm between 2 nodes

1. **Reliable** (perfect) links
2. **Fair-loss** links
3. **Arbitrary** (adversary) links

Retry + de-duplicate

- Could take a long time to get a message
- The message will eventually get through

Cryptographic protocol (i.e.: TLS)

- Note that we can't fix it if the adversary node decided to just drop all messages

2. Node Behavior

Each node executes a specific algorithm

A node can be either

- **Correct:** If it is working as intended (following the algorithm)
- **Faulty:** If it crashes or deviates from the algorithm

2. Node Behavior

Each node executes a specific algorithm

1. **Crash-stop** (fail-stop):
   - A node could crash at any moment (faulty)
   - After crashing, it stops executing forever
2. **Crash-recovery** (fail-recovery):
   - A node may crash at any moment, losing its in-memory state
   - It may resume executing after some time later

2. Node Behavior

Each node executes a specific algorithm

3. **Byzantine** (fail-arbitrary):
   - A node is faulty if it deviates from the algorithm
   - Faulty nodes could do anything, including crashing or malicious behavior

3. Timing Behavior

Nodes communicate over the network

1. **Synchronous**
   - Message latency is no greater than a known upper bound
   - Node executes algorithm at a known speed
2. **Partially Synchronous**
   - The system is asynchronous for some finite (unknown) time
   - Synchronous otherwise

3. Timing Behavior

Nodes communicate over the network

3. **Asynchronous**
   - Messages can be delayed arbitrarily
   - Nodes can pause execution arbitrarily
   - No timing guarantees at all

Note

- "Synchronous" & "Asynchronous" used differently in different CS topics

3. Timing Behavior

Violation of synchrony in practice

- Usually networks have a predictable latency, but occasionally:

1. Message loss requiring retry (fair-loss link)
   - The message could take minutes before getting through
2. Network congestion causing queueing
3. Network/Route reconfiguration

3. Timing Behavior

Violation of synchrony in practice

- Usually nodes execute code at a predictable speed, but occasionally:

1. OS scheduling issues
   - Context switching at a critical moment
2. “Stop the world” Garbage collection pauses
3. Page faults, swap, thrashing

Real-time OS (RTOS) provide scheduling guarantees, but most distributed systems do not use RTOS

System Failure

Concepts

Failure	➔	Service Outage
Availability	➔	Ready to be used immediately
Reliability	➔	Run continuously without failure
Safety	➔	Continue to operate during faults
Maintainability	➔	Ability to repair a failed system

How to achieve high availability?

Failure

- **System** as a **whole** is <u>not working</u>

Fault

- Some **part** of the **system** in <u>not working</u>
  - Node fault: crash or deviating from algorithm
  - Network fault: dropping or significantly delaying messages

Fault tolerance

- **System** as **whole** continues **working** <u>despite faults</u>
  - Some maximum number of faults assumed

Single point of failure

- Node/link whose fault leads to failure

How to achieve high availability?

Failure

- **System** as a **whole** is <u>not working</u>

Fault

- Some **part** of the **system** in <u>not working</u>
  - Node fault: crash or deviating from algorithm
  - Network fault: dropping or significantly delaying messages

Fault tolerance

- **System** as **whole** continues **working** <u>despite faults</u>
  - Some maximum number of faults assumed

Single point of failure

- Node/link whose fault leads to failure

Design a system without a single point of failure

Failure Detector

In order to tolerate a fault, we need to detect it first

Failure detector

- Algorithm that detects if another node is faulty

Perfect failure detector

- Labels a node as faulty if and only if it has crashed (always accurate)

Typical implementation for fail-stop / fail-recovery

- Send a message, await response
- Label as crashed if no reply within some timeout

Failure Detector

In order to tolerate a fault, we need to detect it first

Failure detector

- Algorithm that detects if another node is faulty

Perfect failure detector

- Labels a node as faulty if and only if it has crashed (always accurate)

Typical implementation for fail-stop / fail-recovery

- Send a message, await response
- Label as crashed if no reply within some timeout

Can not differentiate between:

1. Crashed node
2. Temp unresponsive nodes
3. Lost messages
4. Delayed messages

Failure Detector

Perfect timeout-based failure detector exists **only** in a synchronous fail-stop system with reliable links

Eventually perfect failure detector

- May *temporarily* label a node as faulty (even if it is correct)
- May *temporarily* label a node as faulty (even if it has crashed)
- *Eventually*, labels a node as faulty if and only if it has crashed

We could be wrong

Detection not immediate

Replications

Why Replication is Important?

1. Increase Reliability
   - 1 replica crashes ➔ System still works using other replicas
   - 1 replica corrupted ➔ Recover the data using other replicas
2. Improve Performance
   - Distribute workload on multiple servers instead of single server
   - Scale service in terms of geographic area

Node Faults

1. Fail-Stop faults
   - Power outage
   - Detected faults (Checksum error)
2. Fail-recover faults
   - Unresponsive server
   - Network partition
3. Byzantine faults
   - Incorrect software implementation (Bugs)

Node Faults

1. Fail-Stop faults ✓ Replications can handle failures due to fail-stop/fail-recover faults
   - Power outage
   - Detected faults (Checksum error)
2. Fail-recover faults ✓
   - Unresponsive server
   - Network partition
3. Byzantine faults ✗ Replications can not handle failures due to bugs
   - Incorrect software implementation (Bugs)

We always assume that faults are independent:
- Failure at primary replica
- Failure at backup replica

If all servers connected to the same power source?

Replication Cost

1. Resources Needed
   - Replication factor = 2 ➔ Half resources used for replication
   - Replication factor = 3 ➔ 2/3 resources used for replication
2. Network Overhead
   - No replications ➔ 1 write operation needs 1 write request
   - 2 replications ➔ 1 write operation needs at least 2 write requests
   - 3 replications ➔ 1 write operation needs at least 3 write requests

Replication Cost

3. Consistency Overhead
   - Delay/Lag between Primary & Backup? ➔ P/B Sync
   - When to switch over from Primary to Backup? ➔ Cut-over
   - How to handle anomalies till backup takes over?
   - How to create new replicas of the Backup?

When is Replication worth it?

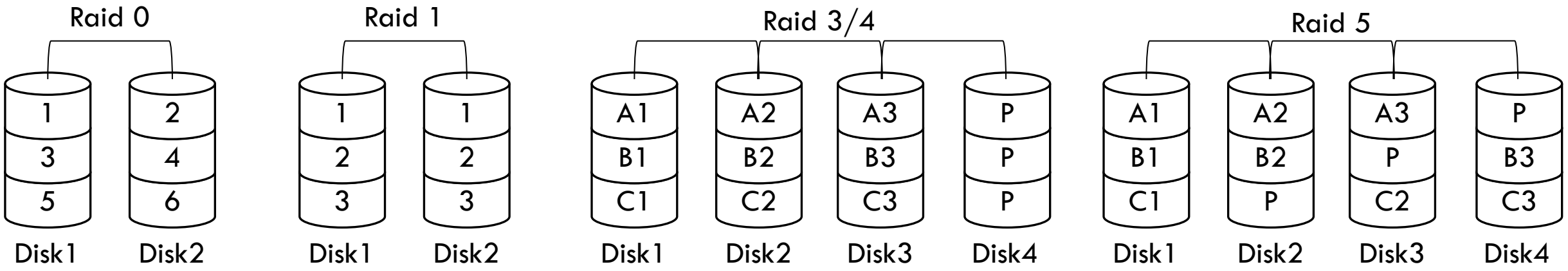
1. Banking System
   - Bank with a single DB server ➔ server fails?
     - No customer access
     - No transactions
     - No revenue
   - Update online bank service?
2. University website?
3. Email server?

What is being Replicated?

1. Request
   - Duplication of client request to the same/different server
2. Data
   - A copy of the data kept at a different location
3. Service
   - A copy of the service provider at a different location
4. Server
   - A mirror of the server at a different location

Basic Example

RAID (Redundant Array of Independent Disks)



Stripping

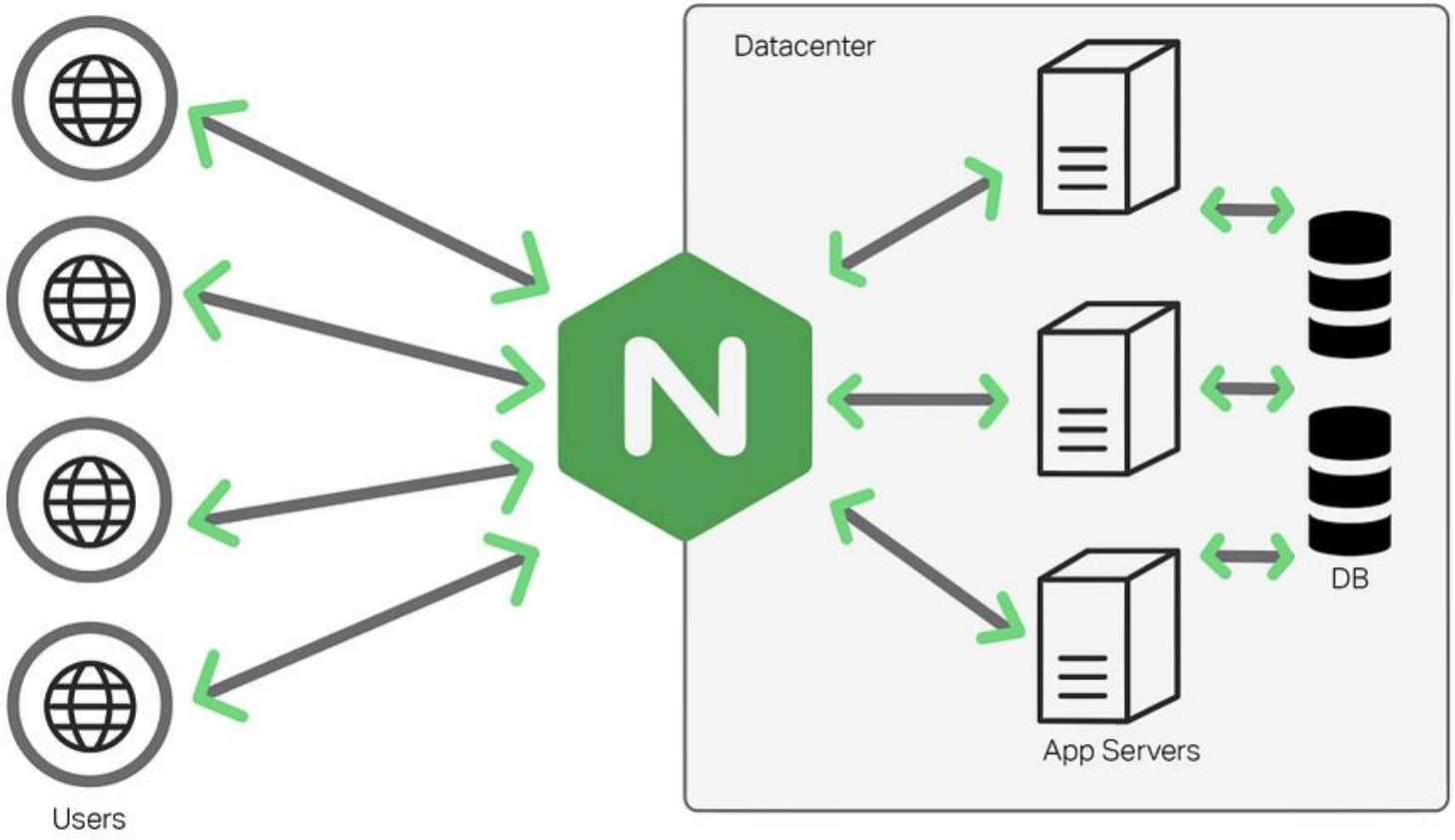
Mirroring

Stripping with dedicated parity

Stripping with distributed parity

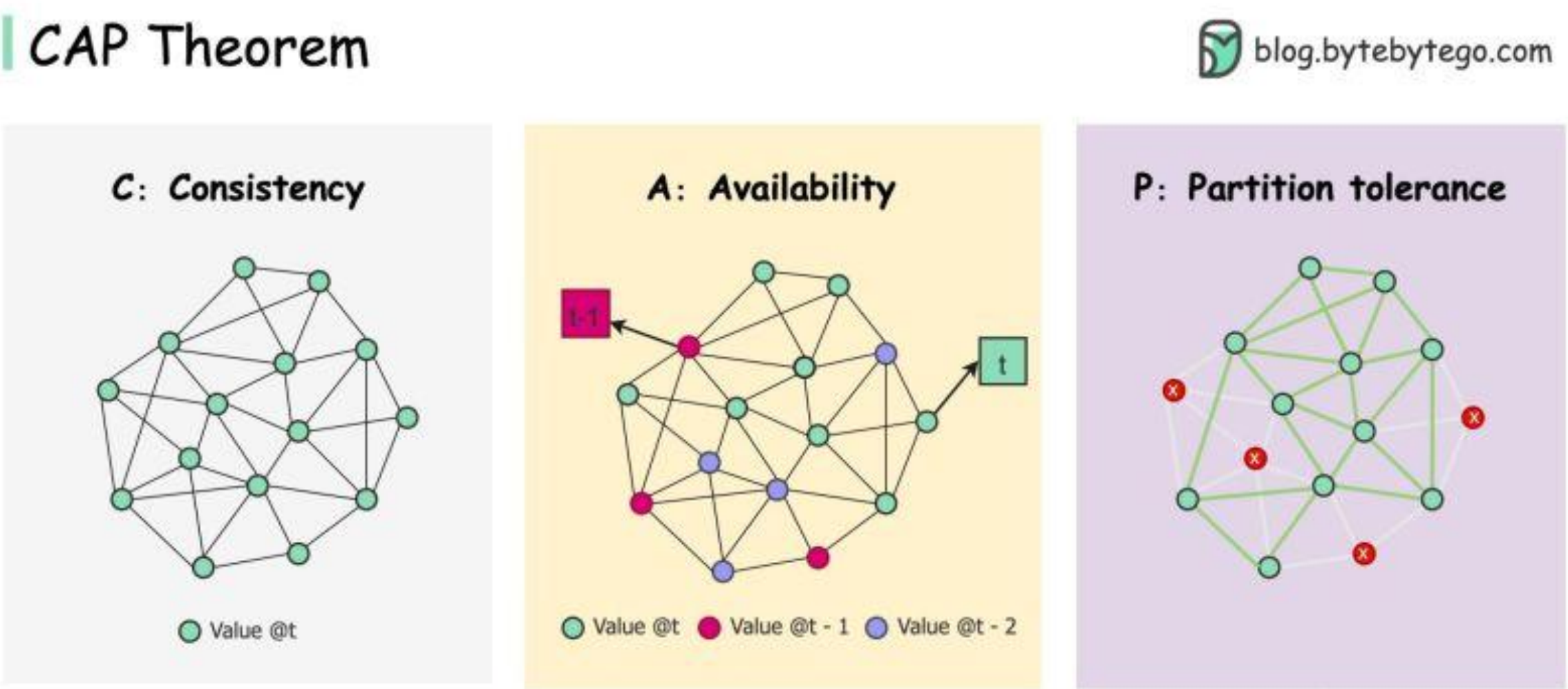
Basic Example

Load Balancing



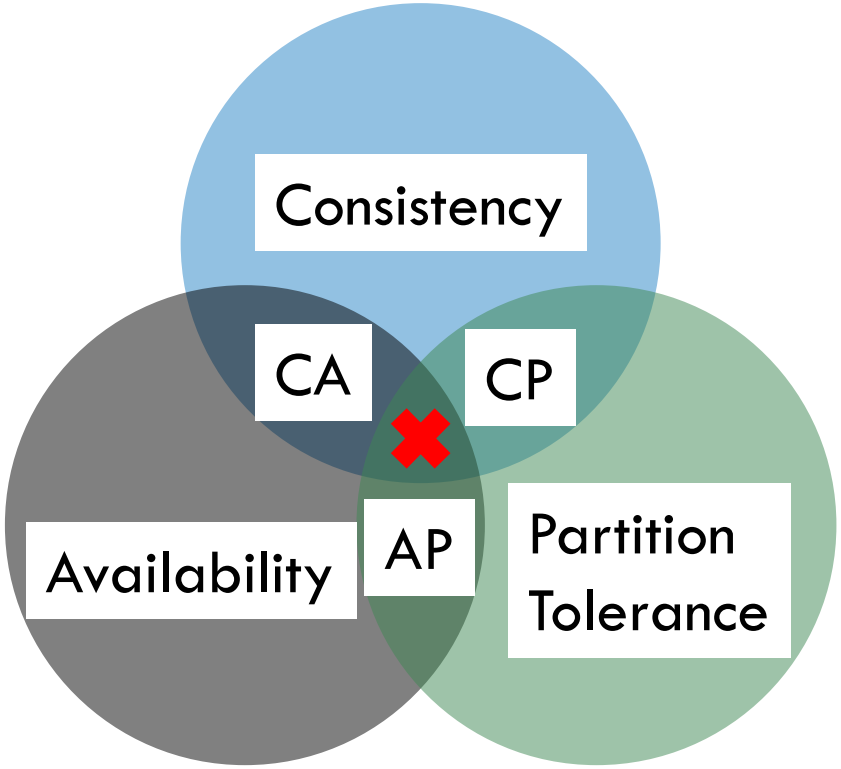
CAP Theory

CAP Theory


CAP Theorem
blog.bytebytego.com
C: Consistency
Value @t
A: Availability
t-1
t
Value @t
Value @t - 1
Value @t - 2
P: Partition tolerance

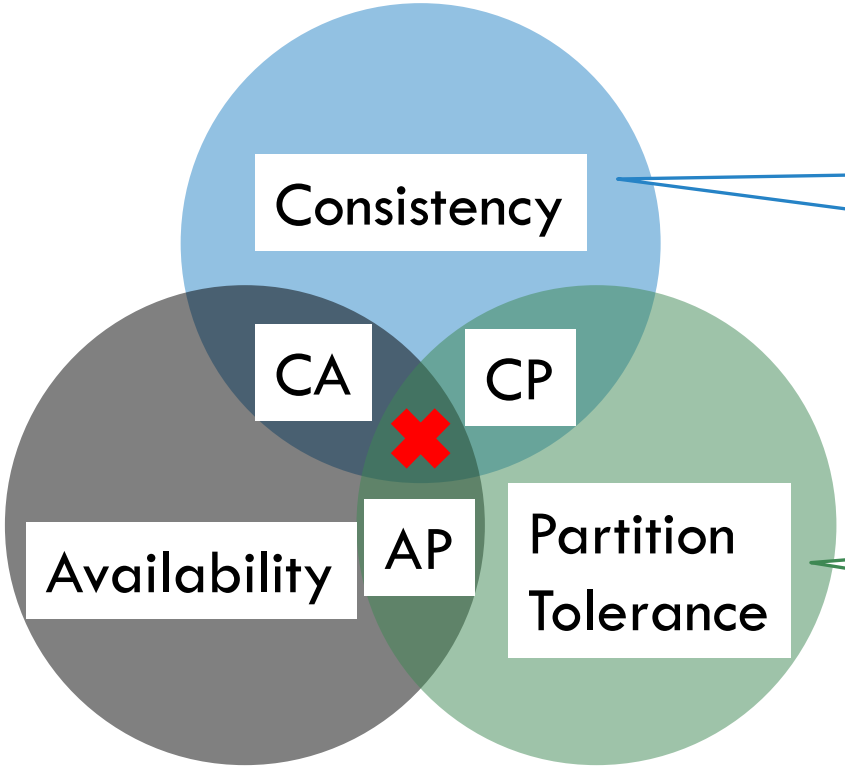
CAP Theory

Any Distributed System can guarantee 2 characteristics only



CAP Theory

Any Distributed System can guarantee 2 characteristics only



All Clients see the **same data** at the same time

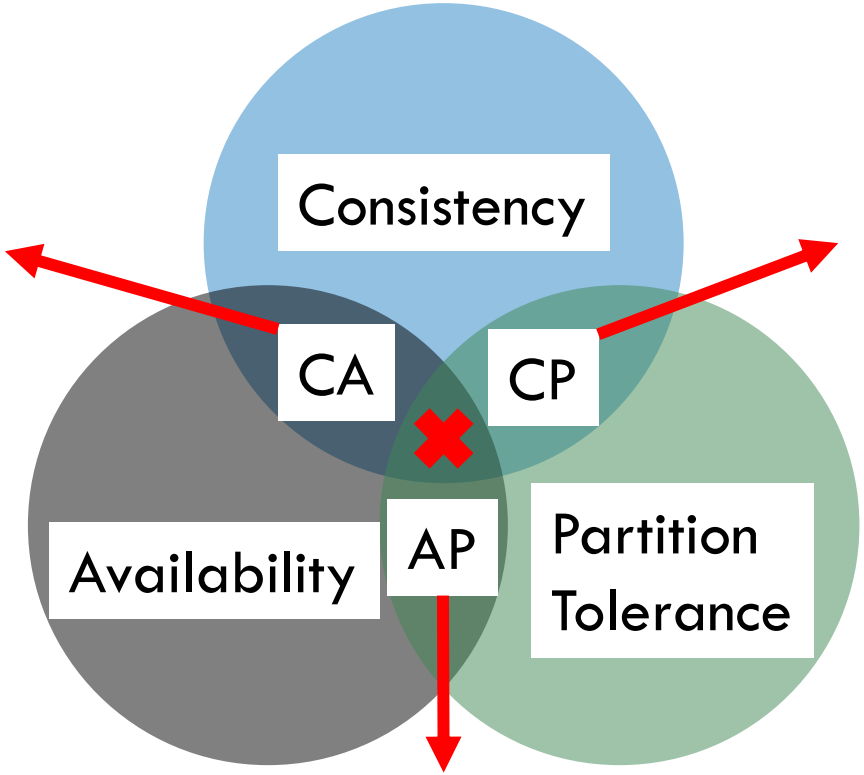
All Clients can **access** the system at **anytime**

System continue to operate despite of **network failures**

CAP Theory

Any Distributed System can guarantee 2 characteristics only

RDBMs:
- SQL Server
- MySQL
- PostgresSQL



Linearizable Service:
- HBase
- Redis
- MongoDB

Eventually Consistent:
- Cassandra
- Dynamo DB

Thank you